



Lab 4: Implementing Nested Procedures and Sorting

Task 1

Translate the C function shown in Listing 1 into RISC-V assembly to calculate the factorial using the iterative approach. How does the stack get populated? Observe and comment on the working of code.

```
1 long long int fact (long long int n){  
2     if (n < 1)  
3         return (1);  
4     else return (n * fact(n - 1));  
5 }
```

Listing 1

Code:

.text

.globl

main:

li x4, 5 # x2=n

li t0, 1

li x1, 1

loop:

ble x4, x1, end

mul t0, t0, x4 # res = res * n

addi x4, x4, -1 # n=n-1

j loop

end:

li x10, 1

add x11, t0, x0

ecall

Output

```
-----  
Starting program C:\Users\Administrator\OneDrive - Habib University\Sem 4\CA\Labs\Lab04\Task1.s  
120Exited with error code 1  
  
Stop program execution!  
-----
```

Task 2

The nth triangular number is like a factorial but instead of a product we use a summation: https://en.wikipedia.org/wiki/Triangular_number

```
1 int ntri (int num){  
2     if (num<=1)  
3         return 1;  
4  
5     return num + ntri (num-1);  
6 }
```

Listing 5: Nth Triangular number

Translate the above C function to calculate the Nth Triangular number into RISC-V assembly. You should also write the wrapper code, i.e., which calls this function with an argument and then displays the result returned by this function.

Since there's recursion going on, you will need to adjust the stack so that you store the return 4 address as well as any useful variables of the caller on the stack before calling the callee. Show the result computed. How does the stack get populated? Observe and comment on the working of code.



Code:

.text

.globl triangle

li a0, 5

triangle:

addi sp, sp, -8

sw ra, 4(sp) # save return address

sw a0, 0(sp)

beq a0, x0, base # base case: if n == 0

addi a0, a0, -1 # n = n - 1

jal triangle # recursive call

lw t0, 0(sp) # restore n

add a0, a0, t0

j done

base:

li a0, 0 # base case result

done:

lw ra, 4(sp)

addi sp, sp, 8

jr ra

j end

end:

j end

Output stack

Address	+0	+1	+2	+3
0x7FFFFFF0	00	00	00	00
0x7FFFFFFEC	00	00	00	00
0x7FFFFFFE8	05	00	00	00
0x7FFFFFFE4	1C	00	00	00
0x7FFFFFFE0	04	00	00	00
0x7FFFFFFDC	1C	00	00	00
0x7FFFFFFD8	03	00	00	00
0x7FFFFFFD4	1C	00	00	00
0x7FFFFFFD0	02	00	00	00
0x7FFFFFFCC	1C	00	00	00
0x7FFFFFFC8	01	00	00	00
0x7FFFFFFC4	1C	00	00	00
0x7FFFFFFC0	00	00	00	00

Code sums the triangle and stores value in a0

x05 (t0)	=	0x00000005
x06 (t1)	=	0x00000000
x07 (t2)	=	0x00000000
x08 (s0)	=	0x00000000
x09 (s1)	=	0x00000000
x10 (a0)	=	0x0000000F
x11 (a1)	=	0x00000000
x12 (a2)	=	0x00000000
x13 (a3)	=	0x00000000

Working:

- Result Computed:
The program calculates the triangular number of 5:
 $5 + 4 + 3 + 2 + 1 = 15$. The final result is returned in register a0.
- Stack Usage in Recursion:
Each recursive call allocates 8 bytes on the stack to save the return address (ra) and the current value of n (a0).
- Stack Population:
As recursion goes deeper ($5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$), each call pushes its own ra and n onto the stack, creating multiple stack frames.
- Unwinding the Stack:
When the base case ($n = 0$) is reached, the function returns 0, and during each return, the saved n is added back to the result while the stack frame is popped.
- Correct Working of Code:
The code correctly preserves caller data across recursive calls using the stack, restores the return address properly, and implements recursion safely with a base case.



Task 3- Bubble Sort

```
1
2 void bubble (int *a, unsigned int len){
3
4     if (a==NULL || len==0)
5         return;
6
7     for(int i=0; i<len; i++){
8         for( int j=i; j<len; j++){
9             if (a[i] < a[j]){
10                 int temp = a[i];
11                 a[i] = a[j];
12                 a[j] = temp;
13             }
14         }
15     }
16
17     return;
18 }
```

Listing 6

Write the equivalent RISC-V assembly code for Listing 6. a and len get passed in x10 and x11 respectively.

Code:

```
.data
arr: .word 7, 2, 9, 4, 1
.text
.globl main
main:
    la a0, arr
    li a1, 5
    li t0, 0
outerloop:
    bge t0, a1, done
    li t1, 0
innerloop:
    addi t2, a1, -1
    sub t2, t2, t0
    bge t1, t2, next_i
    slli t3, t1, 2
    add t4, a0, t3
    lw t5, 0(t4)

    addi t6, t4, 4
    lw x6, 0(t6)

    bge t5, x6, no_swap
    sw x6, 0(t4)
    sw t5, 0(t6)
```



```
no_swap:
    addi t1, t1, 1
    j innerloop
next_i:
    addi t0, t0, 1
    j outerloop
done:
    li a7, 10
    ecall
```

Output

Before sort

0x10000018	00
0x10000014	00
0x10000010	01
0x1000000C	04
0x10000008	09
0x10000004	02
0x10000000	07
0x0FFFFFFC	00
0x0FFFFFF8	00
0x0FFFFFF4	00
0x0FFFFFF0	00
0x0FFFFFEC	00

After sort

0x10000018	00	00
0x10000014	00	00
0x10000010	01	00
0x1000000C	02	00
0x10000008	04	00
0x10000004	07	00
0x10000000	09	00
0x0FFFFFFC	00	00
0x0FFFFFF8	00	00
0x0FFFFFF4	00	00



Task 4 - Student-Selected Program

In this task, students will design and implement a RISC-V assembly program of their own choice that demonstrates the concepts covered in this lab, such as nested procedures, stack management, and control flow.

Students are encouraged to select a program that can be extended or reused in their project. Proper use of procedure calls, register conventions, and stack discipline will be assessed. Students should clearly demonstrate correct execution of the program.

CODE:

GEOMETRIC SERIES:

```
.text
.globl main
main:
    li x1, 7    # nth term
    li x4, 1    # series starts with 1
    li x3, 3    # common ratio
loop:
    beq x1, x0, end    # if n == 0, exit loop
    addi sp, sp, -4
    sw x4, 0(sp)
    mul x4, x4, x3    # x4 = x4 * x3
    addi x1, x1, -1
    j loop
end:
    j end
```

OUTPUT:

0x7FFFFFF0	00
0x7FFFFFFC	01
0x7FFFFFF8	03
0x7FFFFFF4	09
0x7FFFFFF0	1B
0x7FFFFDC	51
0x7FFFFD8	F3
0x7FFFFD4	D9
0x7FFFFD0	00
0x7FFFFCC	00
0x7FFFFC8	00
0x7FFFFC4	00





Assessment Rubric

Lab 4: Implementing Nested Procedures and Sorting

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/10	/5
	Task 2	/10	/5
	Task 3	/15	/10
	Task 4	/15	/10
Total Points: 100		/50	/30
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission / Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.

